Experiment 9: Store Interactions and Visualize History

Aim

To store every prediction interaction (input, result, timestamp) in MongoDB and display a history dashboard using Chart.js for visualization.

Software Requirements

- Python 3.10 or higher
- Flask
- pymongo
- scikit-learn
- MongoDB Atlas
- Chart.js (CDN in HTML)
- HTML/CSS/JS

Theory

Logging interactions allows for later analysis, auditing, and user feedback. MongoDB's flexible schema lets us store arbitrary metadata, while aggregation lets us group/analyze data (e.g., by user, label, date). Chart.js is a JS library for dynamic, interactive charts rendered in the browser.

Procedure

Step 1: Project Setup

```
mkdir exp9_history_dashboard cd
exp9_history_dashboard python -m venv
venv
# Windows:
venv\Scripts\activate #
Mac/Linux:
source venv/bin/activate
```

Step 2: Install Packages

```
pip install flask pymongo tensorflow pillow numpy python-dotenv werkzeug
```

Step 3: MongoDB Atlas Setup

- Create free cluster and DB (`ai_lab`)
- Add IP whitelist (0.0.0.0/0 for dev)
- Get connection string with user and password

Step 4: Directory Structure

```
mkdir templates static static/js static/css
```

Step 5: Example ML Model (reuse or copy mnist_cnn.h5)

```
cp ../exp8_file_upload/mnist_cnn.h5 .
```

Step 6: Create .env file

Create a `.env` file containing the (Refer to previous experiments for setup and getting the url)

```
MONGO_URI=mongodb+srv://admin:YOUR_PASSWORD@cluster0.xxxxx.mongodb.net/
```

Important

Above variable is an example. Replace with the value given in MongoDB atlas

Step 6: Flask App with Logging (app.py)

```
from flask import Flask, render_template, request, jsonify from
pymongo import MongoClient
from werkzeug.utils import secure_filename
import os
from datetime import datetime
from PIL import Image
from dotenv import load_dotenv
import numpy as np
from tensorflow.keras.models import load_model

load_dotenv()

app = Flask(__name__)
app.config['UPLOAD_FOLDER'] =
'static/uploads'
os.makedirs(app.config['UPLOAD_FOLDER'],  exist_ok=True)

mongo_uri =
os.getenv("MONGO_URI") client
= MongoClient(mongo_uri) db =
client["ai_lab"]
history = db["prediction_history"]
```

```
model =

load_model("mnist_cnn.h5") def

preprocess_image(path):
    # 1) load grayscale
    img = Image.open(path).convert('L')

    # 2) make square by padding preserving aspect ratio
    w,h = img.size
    sz = max(w,h)
    padded = Image.new('L', (sz,sz), color=255)  # white background padded.paste(img, ((sz-w)//2, (sz-h)//2))

    # 3) resize to 28x28
    small = padded.resize((28,28), Image.LANCZOS)
    # 4) invert if background is dark/bright mismatch arr =
```

```
    np.array(small).astype(np.float32)
    # Determine if digit is dark-on-light (common for photos) -> invert to MNIST style
(white digit on black)
    if arr.mean() > 127: arr
        = 255.0 - arr
    # 5) normalize to 0-1 and reshape
    arr = arr / 255.0
    arr = arr.reshape(1,28,28,1).astype(np.float32) return
    arr
@app.route('/')
def index():
    return render_template('dashboard.html')
@app.route('/predict', methods=['POST'])
def predict():
    if 'file' not in request.files:
        return jsonify(error="no file"), 400 f =
    request.files['file']
    filename = secure_filename(f.filename)
    path = os.path.join(app.config['UPLOAD_FOLDER'], filename)
    f.save(path)
    X = preprocess_image(path)
    probs = model.predict(X)[0]
    pred = int(np.argmax(probs))
    conf = float(np.max(probs))
    ts = datetime.now()
    history.insert_one({"filename": filename, "prediction": pred, "timestamp": ts}) return
    jsonify(prediction=pred, confidence=conf, filename=filename)
@app.route('/api/history') def
api_history():
    docs = list(history.find().sort([('timestamp', -1)]).limit(30))
    for d in docs:
        d['_id'] = str(d['_id'])
        d['timestamp'] = d['timestamp'].strftime('%Y-%m-%d
    %H:%M') return jsonify(docs)
if __name__ == '__main__':
    os.makedirs('static/uploads', exist_ok=True)
    app.run(debug=True)
```

Step 7: HTML Dashboard (templates/dashboard.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Prediction History Dashboard</title>
    <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
    <style>body{font-family:Arial;background:#f3f3ff;padding:40px;} #main{background:white;
border-radius:10px; padding:40px; max-width:700px;margin:auto;}</style>
</head>
<body>
<div id="main">
    <h2>Digit Recognition - History Dashboard</h2>
    <form id="uploadForm" enctype="multipart/form-data">
        <input type="file" name="file" accept="image/*" required>
        <button type="submit">Predict</button>
    </form>
    <div id="result"></div>
    <h3>Recent Predictions</h3>
    <table id="historyTable" border="1" cellpadding="4" style="width:100%;margin-
bottom:20px;">
        <thead><tr><th>Img</th><th>Prediction</th><th>Time</th></tr></thead>
        <tbody></tbody>
    </table>
    <h3>Digit Distribution</h3>
    <canvas id="digitChart" width="400" heightght="120"></canvas>
</div>
<script>
async function getHistory() {
    const resp = await fetch('/api/history'); const
    data = await resp.json();
    let tbl = ''; let
    counts = {};
    data.forEach(r => {
        tbl += `<tr><td><img src='/static/uploads/${r.filename}' width=28></td>
<td>${r.prediction}</td><td>${r.timestamp}</td></tr>`;
        counts[r.prediction] = (counts[r.prediction]||0)+1;
    });
    document.querySelector('#historyTable tbody').innerHTML = tbl;
    // Chart
    let ctx = document.getElementById('digitChart').getContext('2d');
    if(window.dChart) window.dChart.destroy();
    window.dChart = new Chart(ctx, {
        type:'bar',
```

```
            data:{labels:Object.keys(counts),datasets: [{label:'Count',data:Object.values(counts)}]},
            options:{scales:{y:{beginAtZero:true}}}
        });
    }
    document.getElementById('uploadForm').onsubmit = async function(e){
        e.preventDefault();
        const formData = new FormData(e.target);
        const res = await fetch('/predict',{method:'POST',body:formData}); const
        out = await res.json(); document.getElementById('result').innerHTML =
        out.prediction!==undefined
            ? `<b>Predicted: ${out.prediction}</b><br><img src='/static/uploads/${out.filename}' width=80>`
            : `<span style=color:red>${out.error}</span>`;
        getHistory();
    };
    getHistory();
    </script>
    </body>
    </html>
```

Step 8: Run & Test

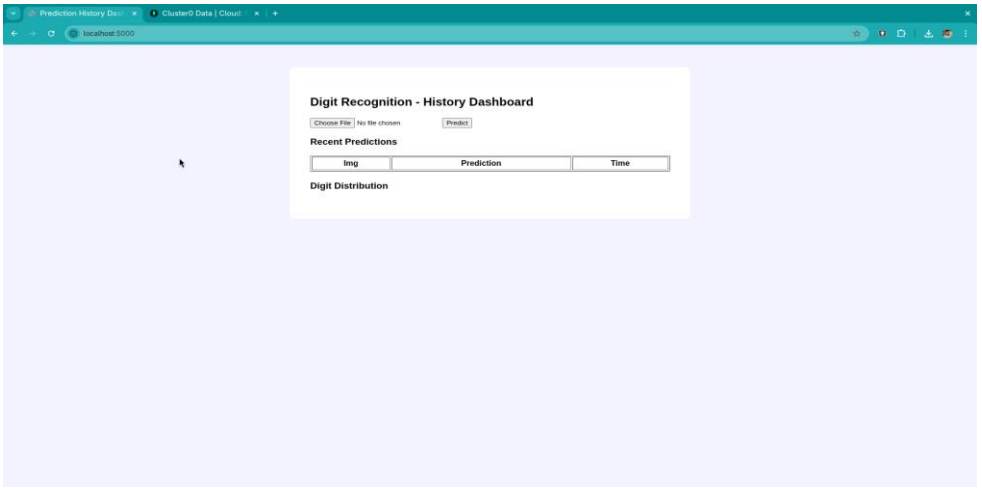
- Run: `python app.py`
- Upload images, see instant prediction and history
- View bar chart for most common digits in last 30 predictions

Output

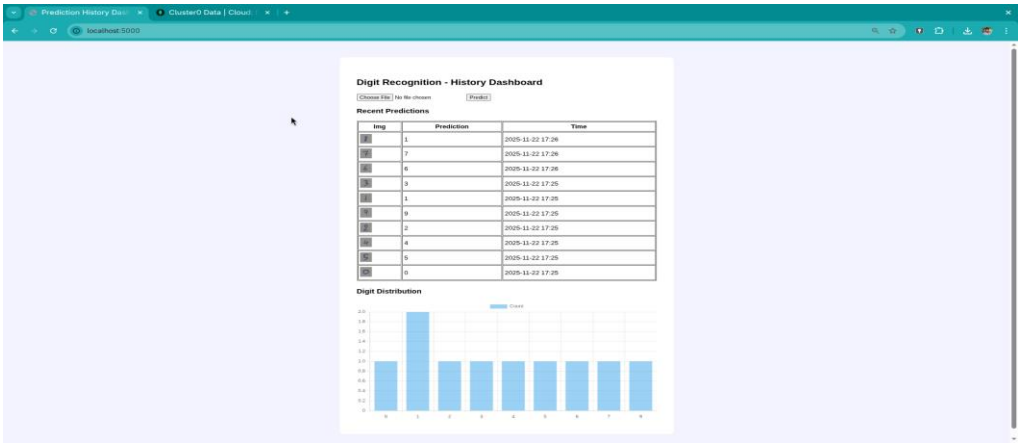
- Uploaded images and their predictions in a table
- Bar chart of digit prediction frequency, updating in real time with each upload

Screen-shots

Default Page:



After Predictions



Important

The model is not 100% accurate, so possible for false values. In above screen shot, 8 is mis-labeled.

Result

A complete full-stack system where every ML prediction is stored, tracked, and visualized via interactive dashboard.